

Q1. Jamboree Education — Linear Regression

Predicting Graduate Admission Probability

Name: Preetham Teja Ambati

Objective: Analyse what factors influence graduate admission chances and build a Linear Regression model (with Ridge/Lasso comparison) that predicts an applicant's `Chance of Admit`, testing all standard linear regression assumptions along the way.

```
# ---- Imports ----
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# statsmodels gives us the classic OLS summary table (coef / std err / t / P>|t|).
# Fall back to a manual (mathematically identical) OLS implementation if it
# isn't installed in this environment -- Google Colab has it pre-installed.
try:
    import statsmodels.api as sm
    from statsmodels.stats.diagnostic import het_breuschpagan
    from statsmodels.stats.outliers_influence import variance_inflation_factor
    HAS_SM = True
except ImportError:
    HAS_SM = False
    print("statsmodels not found -- using an equivalent manual OLS implementation.")

sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 100
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

print("HAS_SM =", HAS_SM)
```

HAS_SM = True

1. Problem Definition & Exploratory Data Analysis (10 points)

Problem: Jamboree wants a model that estimates an applicant's probability of graduate admission from GRE, TOEFL, University Rating, SOP, LOR, CGPA and Research experience, and wants to understand *which* of these factors matter most and how they relate to each other -- so Jamboree can give applicants targeted, actionable advice.

This is a supervised regression problem: the target `Chance of Admit` is continuous, bounded in [0, 1].

```
df = pd.read_csv("https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/001/839/original/Jamboree_Admission.csv")
df.columns = df.columns.str.strip() # 'LOR ' has a trailing space in the raw file
print("Shape:", df.shape)
df.head()
```

Shape: (500, 9)

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	1	337	118	4	4.5	4.5	9.65	1	0.92
1	2	324	107	4	4.0	4.5	8.87	1	0.76
2	3	316	104	3	3.0	3.5	8.00	1	0.72
3	4	322	110	3	3.5	2.5	8.67	1	0.80
4	5	314	103	2	2.0	3.0	8.21	0	0.65



Next steps: [Generate code with df](#) [New interactive sheet](#)

```
print(df.head().to_string())
```

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	1	337	118	4	4.5	4.5	9.65	1	0.92
1	2	324	107	4	4.0	4.5	8.87	1	0.76
2	3	316	104	3	3.0	3.5	8.00	1	0.72
3	4	322	110	3	3.5	2.5	8.67	1	0.80
4	5	314	103	2	2.0	3.0	8.21	0	0.65

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Serial No.            500 non-null    int64
1   GRE Score              500 non-null    int64
2   TOEFL Score            500 non-null    int64
3   University Rating      500 non-null    int64
4   SOP                    500 non-null    float64
5   LOR                    500 non-null    float64
6   CGPA                   500 non-null    float64
7   Research                500 non-null    int64
8   Chance of Admit        500 non-null    float64
dtypes: float64(4), int64(5)
memory usage: 35.3 KB
```

```
# Drop the unique row identifier -- it carries no predictive signal
df = df.drop(columns=["Serial No."])
print("Columns after dropping Serial No.:", list(df.columns))
```

```
Columns after dropping Serial No.: ['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR', 'CGPA', 'Research', 'Chance
```

```
print("Missing values per column:")
print(df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
```

```
Missing values per column:
GRE Score      0
TOEFL Score    0
University Rating  0
SOP            0
LOR            0
CGPA           0
Research       0
Chance of Admit  0
dtype: int64
```

```
Duplicate rows: 0
```

```
print(df.describe().T.round(3))
```

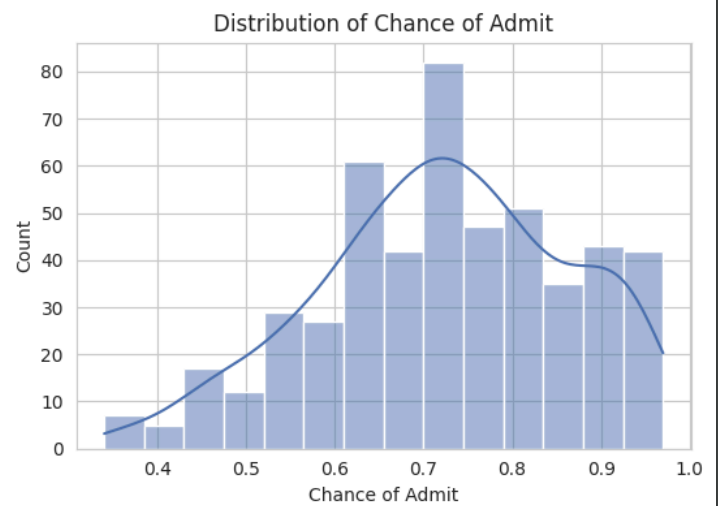
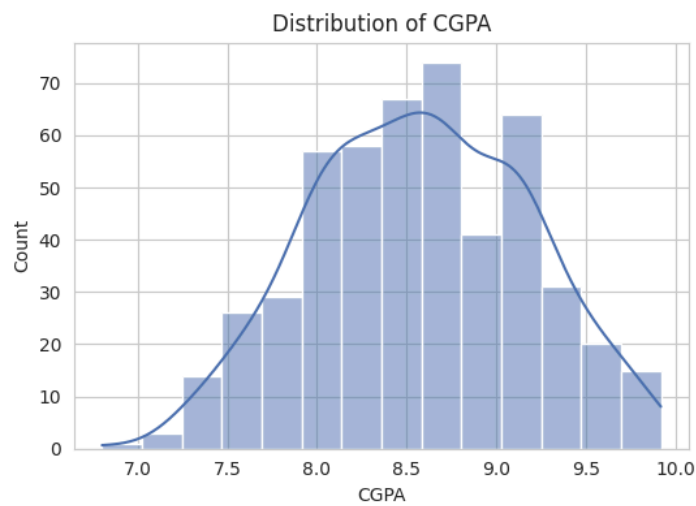
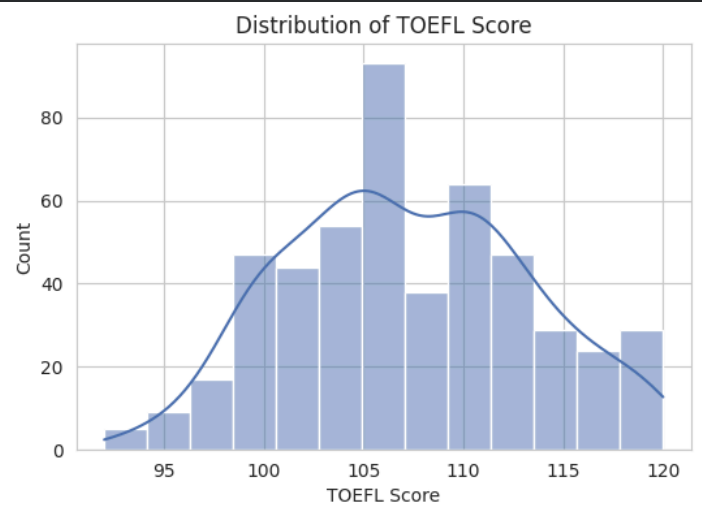
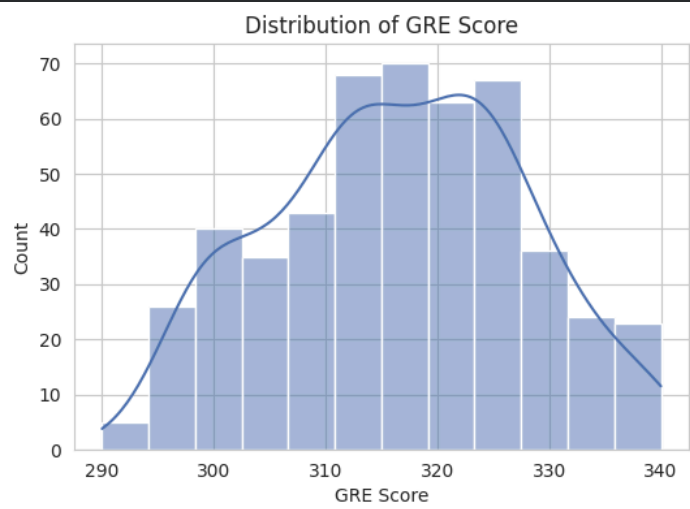
	count	mean	std	min	25%	50%	75%	max
GRE Score	500.0	316.472	11.295	290.00	308.000	317.00	325.00	340.00
TOEFL Score	500.0	107.192	6.082	92.00	103.000	107.00	112.00	120.00
University Rating	500.0	3.114	1.144	1.00	2.000	3.00	4.00	5.00
SOP	500.0	3.374	0.991	1.00	2.500	3.50	4.00	5.00
LOR	500.0	3.484	0.925	1.00	3.000	3.50	4.00	5.00
CGPA	500.0	8.576	0.605	6.80	8.128	8.56	9.04	9.92
Research	500.0	0.560	0.497	0.00	0.000	1.00	1.00	1.00
Chance of Admit	500.0	0.722	0.141	0.34	0.630	0.72	0.82	0.97

Observations: 500 rows, 9 columns originally (8 after dropping the ID). No missing values, no duplicates -- no imputation needed. GRE Score, TOEFL Score, University Rating and Research are integers; SOP, LOR, CGPA and Chance of Admit are floats. University Rating, SOP, LOR are ordinal (1-5) and Research is binary -- all kept as numeric since a linear model can use ordinal scales directly.

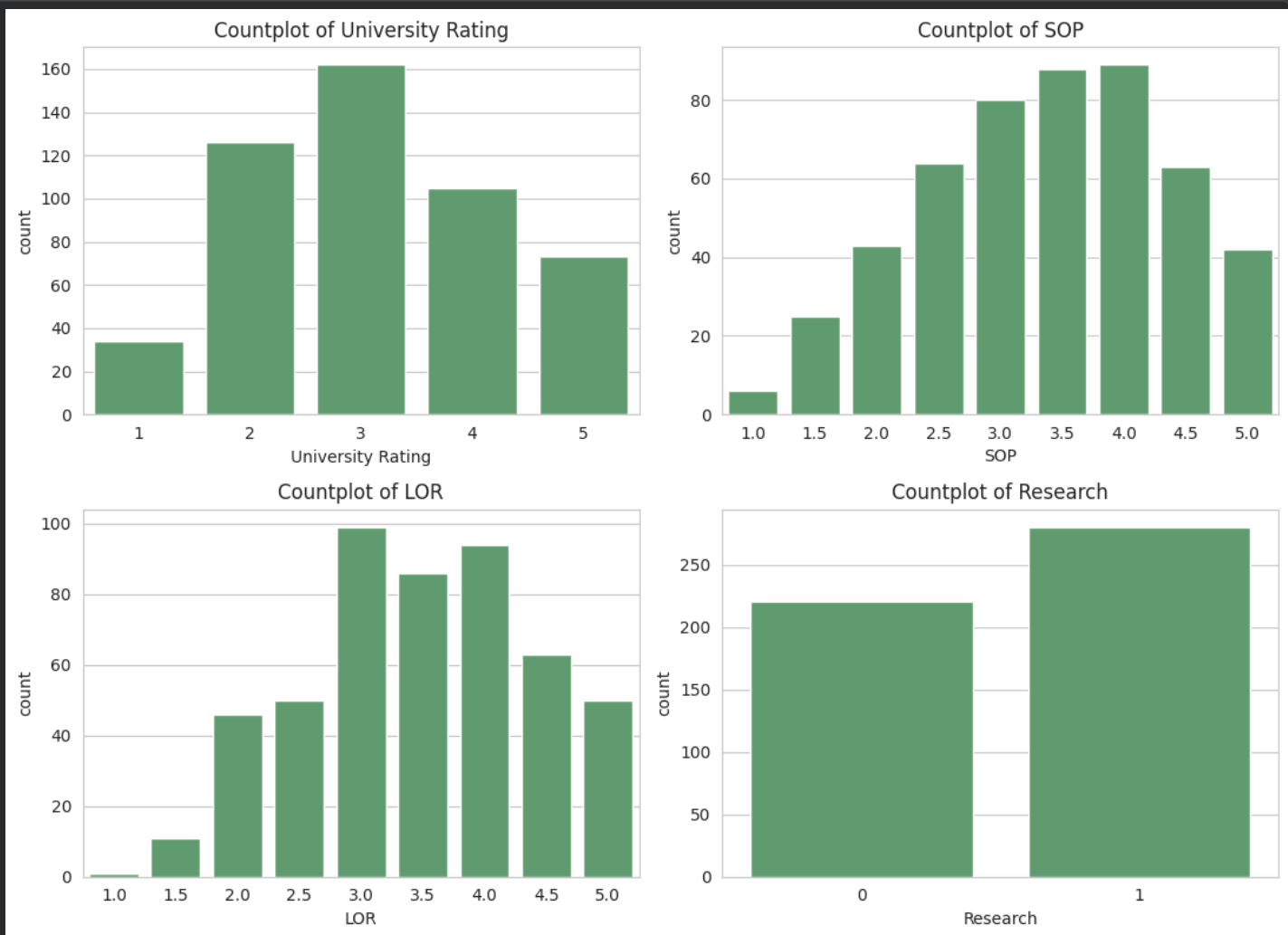
GRE ranges 290-340, TOEFL 92-120, CGPA 6.8-9.92 -- a wide, realistic spread of applicant merit, not just top scorers.

1.1 Univariate Analysis

```
cont_cols = ["GRE Score", "TOEFL Score", "CGPA", "Chance of Admit"]
fig, axes = plt.subplots(2, 2, figsize=(11, 8))
for ax, col in zip(axes.flatten(), cont_cols):
    sns.histplot(df[col], kde=True, ax=ax, color="#4C72B0")
    ax.set_title(f"Distribution of {col}")
plt.tight_layout()
plt.show()
```



```
cat_cols = ["University Rating", "SOP", "LOR", "Research"]
fig, axes = plt.subplots(2, 2, figsize=(11, 8))
for ax, col in zip(axes.flatten(), cat_cols):
    sns.countplot(x=df[col], ax=ax, color="#55A868")
    ax.set_title(f"Countplot of {col}")
plt.tight_layout()
plt.show()
```

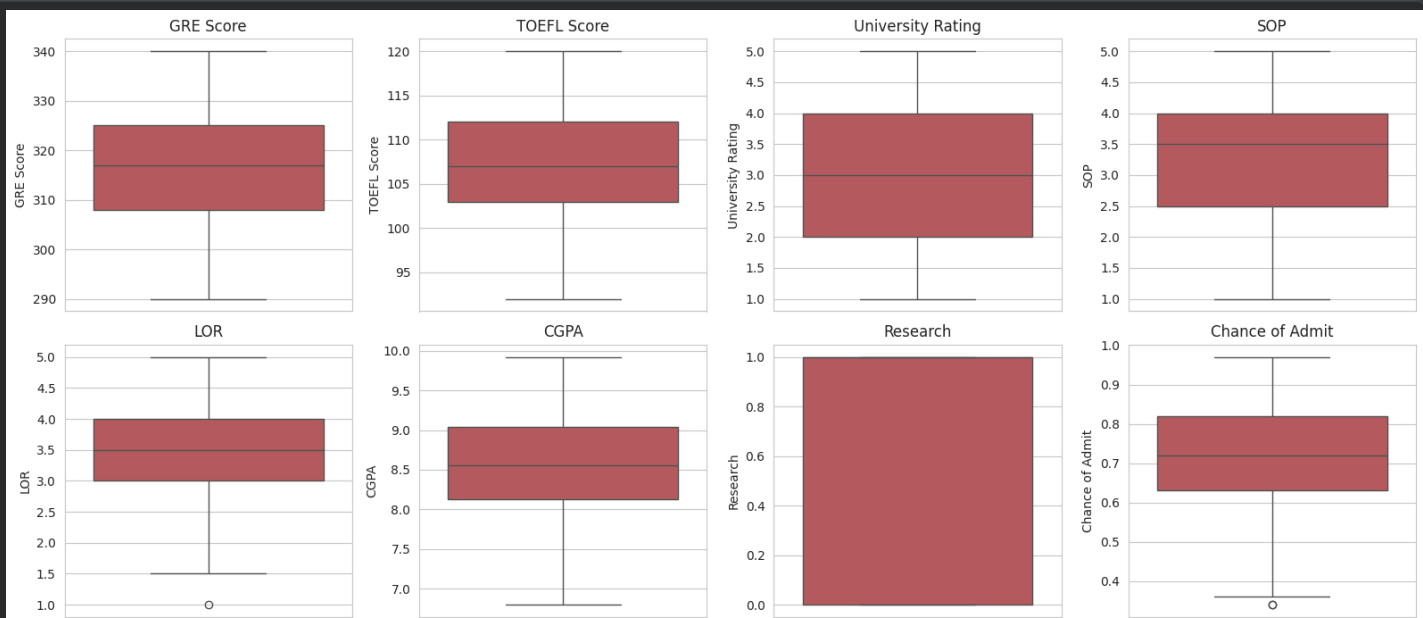


Comment: Continuous variables (GRE, TOEFL, CGPA, Chance of Admit) are roughly symmetric / mildly left-skewed with no long tails. University Rating, SOP and LOR are concentrated around 3, and about 56% of applicants have research experience.

1.2 Outlier Detection

```
allnum = ["GRE Score", "TOEFL Score", "University Rating", "SOP", "LOR", "CGPA", "Research", "Chance of Admit"]
fig, axes = plt.subplots(2, 4, figsize=(16, 7))
for ax, col in zip(axes.flatten(), allnum):
    sns.boxplot(y=df[col], ax=ax, color="#C44E52")
    ax.set_title(col)
plt.tight_layout()
plt.show()

# Quantify outliers with the 1.5*IQR rule
print("Outlier counts (1.5xIQR rule):")
for col in allnum:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n_out = ((df[col] < low) | (df[col] > high)).sum()
    print(f" {col:20s}: {n_out}")
```



Outlier counts (1.5xIQR rule):

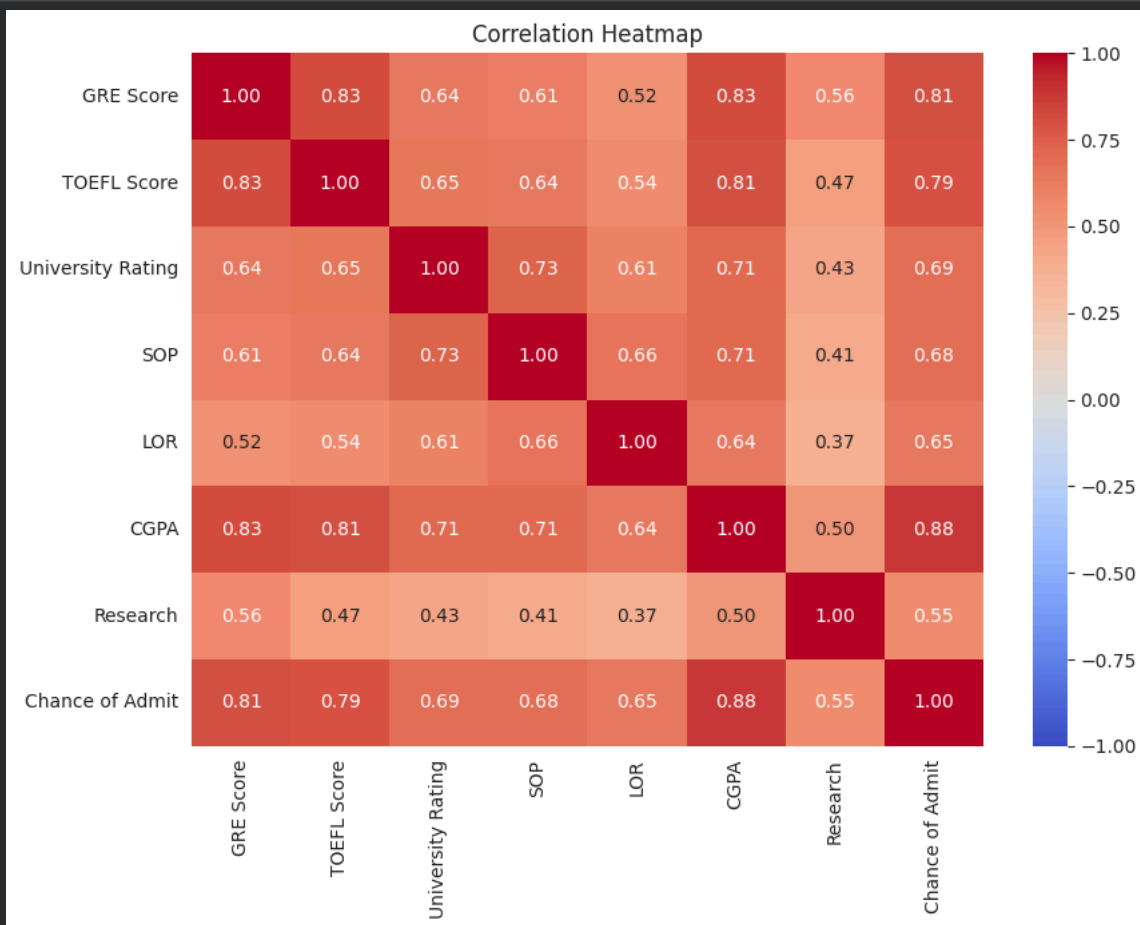
```
GRE Score      : 0
TOEFL Score    : 0
University Rating : 0
SOP            : 0
LOR            : 1
CGPA           : 0
Research       : 0
Chance of Admit : 2
```

Comment: Almost no statistical outliers. The one or two flagged points (a low LOR, two low Chance-of-Admit values) are legitimate low-merit applicants, not data errors -- no impossible values exist (GRE>340, CGPA>10, Admit outside [0,1]). **No rows are removed.**

1.3 Bivariate Analysis & Correlation

```
plt.figure(figsize=(9, 7))
corr = df.corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

print("\nCorrelation with Chance of Admit (sorted):")
print(corr["Chance of Admit"].sort_values(ascending=False))
```



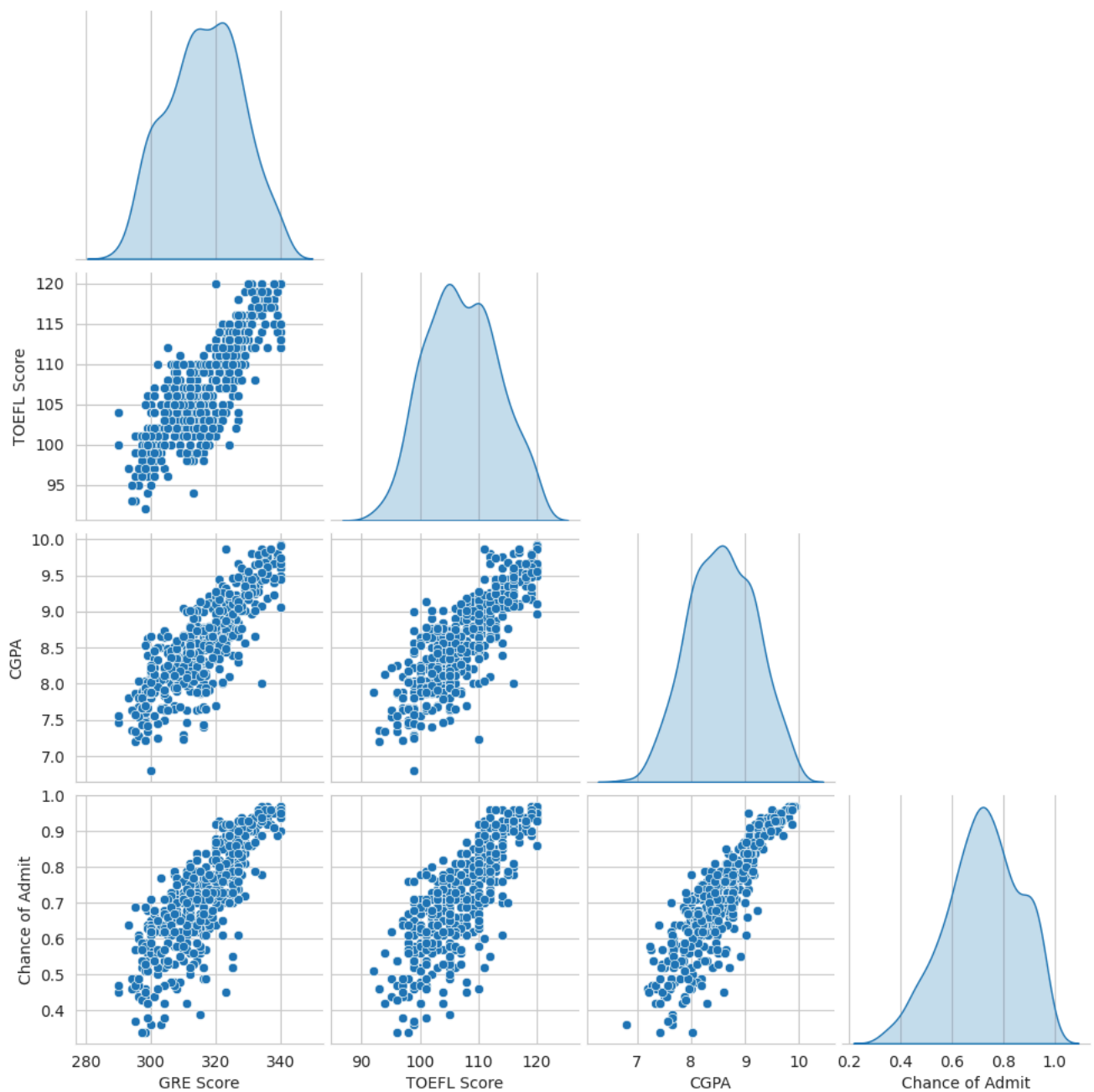
Correlation with Chance of Admit (sorted):

```
Chance of Admit    1.000000
CGPA                0.882413
GRE Score          0.810351
TOEFL Score        0.792228
University Rating   0.690132
SOP                0.684137
LOR                0.645365
Research           0.545871
```

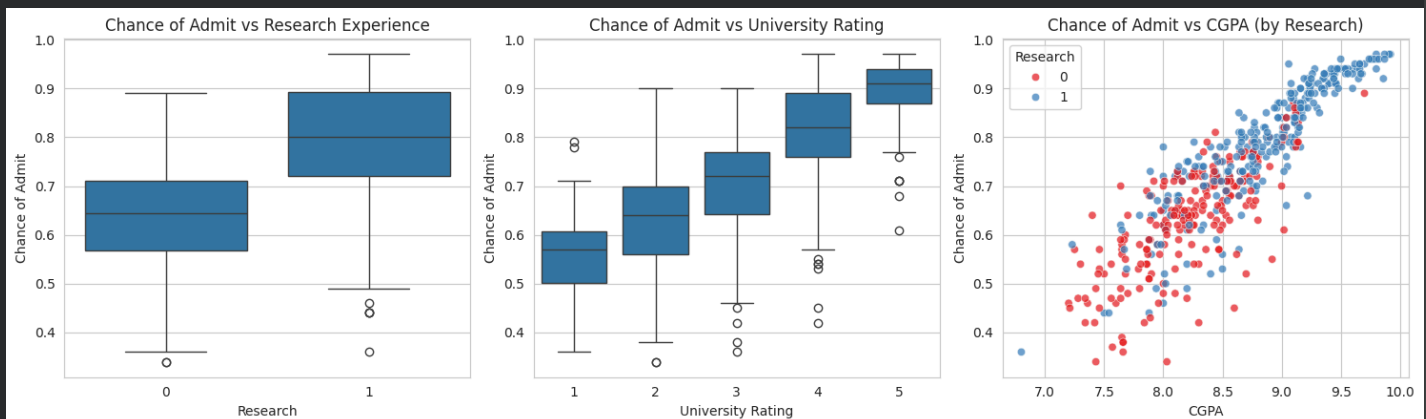
Name: Chance of Admit, dtype: float64

```
sns.pairplot(df[["GRE Score", "TOEFL Score", "CGPA", "Chance of Admit"]], corner=True, diag_kind="kde")
plt.suptitle("Pairplot: Key Continuous Variables", y=1.02)
plt.show()
```

Pairplot: Key Continuous Variables



```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
sns.boxplot(x="Research", y="Chance of Admit", data=df, ax=axes[0])
axes[0].set_title("Chance of Admit vs Research Experience")
sns.boxplot(x="University Rating", y="Chance of Admit", data=df, ax=axes[1])
axes[1].set_title("Chance of Admit vs University Rating")
sns.scatterplot(x="CGPA", y="Chance of Admit", hue="Research", data=df, ax=axes[2], palette="Set1", alpha=0.7)
axes[2].set_title("Chance of Admit vs CGPA (by Research)")
plt.tight_layout()
plt.show()
```



Insight: Every predictor is positively correlated with the target. **CGPA is the strongest driver** ($r \approx 0.88$), followed by GRE (≈ 0.81) and TOEFL (≈ 0.79). University Rating, SOP, LOR are moderate (≈ 0.65 - 0.69), Research is weakest (≈ 0.55) but still shows a clear positive shift in the boxplot. GRE/TOEFL/CGPA are also correlated *with each other* (0.83 - 0.88) -- flagged for the VIF check later. Applicants with research experience show a visibly higher median admit chance even controlling for CGPA.

2. Data Preprocessing (10 points)

Duplicate check: done above -> 0 duplicates. **Missing values:** done above -> none. **Outlier treatment:** as discussed, the few flagged points are genuine low-merit applicants, not errors -- capping them would distort the low end of the target's range the model needs to learn from, so **no capping/removal was applied**. **Feature engineering:** `Serial No.` already dropped. Ordinal columns (University Rating, SOP, LOR) are kept on their natural numeric scale (not one-hot encoded) since that preserves their ordering, which one-hot encoding would discard, and the pairplot/correlation above already show strong linear relationships -- no interaction/polynomial terms are needed for a first-cut linear model.

```
X = df.drop(columns=["Chance of Admit"])
y = df["Chance of Admit"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Train shape:", X_train.shape, " Test shape:", X_test.shape)

Train shape: (400, 7)  Test shape: (100, 7)
```

3. Model Building (10 points)

3.1 Full OLS Model — all 7 predictors

```
def manual_ols(X_df, y_ser):
    """OLS via normal equations -- mathematically identical to statsmodels OLS."""
    Xd = np.column_stack([np.ones(len(X_df)), X_df.values])
    n, k = Xd.shape
    beta, _, _, _ = np.linalg.lstsq(Xd, y_ser.values, rcond=None)
    y_hat = Xd @ beta
    resid = y_ser.values - y_hat
    ss_res = np.sum(resid ** 2)
    ss_tot = np.sum((y_ser.values - y_ser.values.mean()) ** 2)
    r2 = 1 - ss_res / ss_tot
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - k)
    dof = n - k
    mse = ss_res / dof
    xtx_inv = np.linalg.inv(Xd.T @ Xd)
    se = np.sqrt(np.diag(mse * xtx_inv))
    t_stats = beta / se
    p_values = 2 * (1 - stats.t.cdf(np.abs(t_stats), dof))
    f_stat = (r2 / (k - 1)) / ((1 - r2) / dof)
    f_pvalue = 1 - stats.f.cdf(f_stat, k - 1, dof)
    names = ["const"] + list(X_df.columns)
    summary = pd.DataFrame({"coef": beta, "std err": se, "t": t_stats, "P>|t|": p_values}, index=names)
    return summary, r2, adj_r2, f_stat, f_pvalue, resid, y_hat

if HAS_SM:
    X_train_sm = sm.add_constant(X_train)
    ols_model = sm.OLS(y_train, X_train_sm).fit()
    print(ols_model.summary())
    resid_full = ols_model.resid.values
    yhat_full = ols_model.fittedvalues.values
    r2_full, adjr2_full = ols_model.rsquared, ols_model.rsquared_adj
else:
    ols_summary, r2_full, adjr2_full, f_full, fp_full, resid_full, yhat_full = manual_ols(X_train, y_train)
    print(ols_summary.round(4))
    print(f"\nR2={r2_full:.4f}  Adj R2={adjr2_full:.4f}  F-stat={f_full:.2f}  Prob(F)={fp_full:.4g}  n={len(y_train)}")
```

OLS Regression Results						
=====						
Dep. Variable:	Chance of Admit	R-squared:	0.821			
Model:	OLS	Adj. R-squared:	0.818			
Method:	Least Squares	F-statistic:	257.0			
Date:	Wed, 12 Aug 2026	Prob (F-statistic):	3.41e-142			
Time:	18:50:41	Log-Likelihood:	561.91			
No. Observations:	400	AIC:	-1108.			
Df Residuals:	392	BIC:	-1076.			
Df Model:	7					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-1.4214	0.123	-11.549	0.000	-1.663	-1.179
GRE Score	0.0024	0.001	4.196	0.000	0.001	0.004
TOEFL Score	0.0030	0.001	3.174	0.002	0.001	0.005
University Rating	0.0026	0.004	0.611	0.541	-0.006	0.011

SOP	0.0018	0.005	0.357	0.721	-0.008	0.012
LOR	0.0172	0.005	3.761	0.000	0.008	0.026
CGPA	0.1125	0.011	10.444	0.000	0.091	0.134
Research	0.0240	0.007	3.231	0.001	0.009	0.039

Omnibus:	86.232	Durbin-Watson:	2.050
Prob(Omnibus):	0.000	Jarque-Bera (JB):	190.099
Skew:	-1.107	Prob(JB):	5.25e-42
Kurtosis:	5.551	Cond. No.	1.37e+04

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.37e+04. This might indicate that there are strong multicollinearity or other numerical problems.

Reading the output: CGPA, GRE Score, TOEFL Score, LOR and Research all have $p < 0.01$ (statistically significant).

University Rating ($p \approx 0.54$) and SOP ($p \approx 0.72$) are NOT significant once CGPA/GRE/TOEFL are already in the model -- their signal is absorbed by these stronger, correlated predictors. $R^2 \approx 0.82$, Adj $R^2 \approx 0.82$: the model explains ~82% of the variance in Chance of Admit.

3.2 Model coefficients (named)

```
if HAS_SM:
    coefs = ols_model.params.drop("const")
else:
    coefs = ols_summary["coef"].drop("const")
print(coefs.sort_values(ascending=False).round(4))
```

CGPA	0.1125
Research	0.0240
LOR	0.0172
TOEFL Score	0.0030
University Rating	0.0026
GRE Score	0.0024
SOP	0.0018
dtype:	float64

3.3 Ridge and Lasso Regression

```
alphas = np.logspace(-4, 2, 100)
ridge = RidgeCV(alphas=alphas, cv=5).fit(X_train, y_train)
lasso = LassoCV(alphas=alphas, cv=5, max_iter=10000).fit(X_train, y_train)

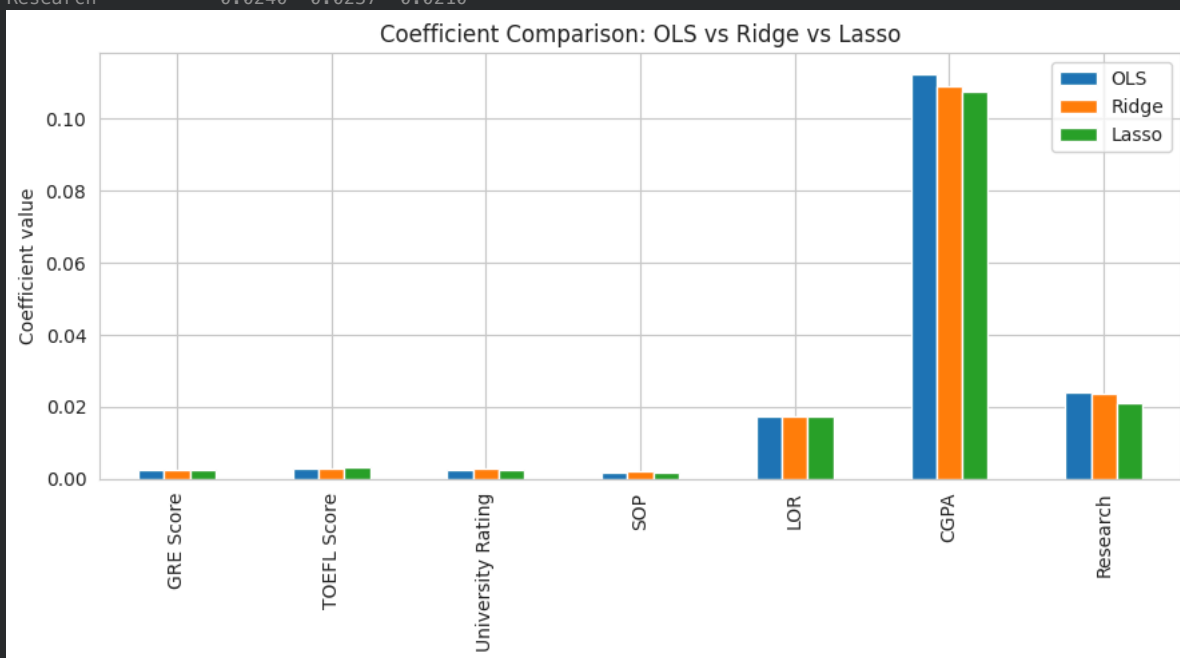
print("Ridge best alpha:", ridge.alpha_)
print("Lasso best alpha:", lasso.alpha_)

coef_df = pd.DataFrame({
    "OLS": coefs.values,
    "Ridge": ridge.coef_,
    "Lasso": lasso.coef_
}, index=X_train.columns)
print(coef_df.round(4))

coef_df.plot(kind="bar", figsize=(9, 5))
plt.title("Coefficient Comparison: OLS vs Ridge vs Lasso")
plt.ylabel("Coefficient value")
plt.tight_layout()
plt.show()
```

Ridge best alpha: 1.0
 Lasso best alpha: 0.0005336699231206312

	OLS	Ridge	Lasso
GRE Score	0.0024	0.0025	0.0027
TOEFL Score	0.0030	0.0031	0.0031
University Rating	0.0026	0.0028	0.0027
SOP	0.0018	0.0022	0.0018
LOR	0.0172	0.0175	0.0172
CGPA	0.1125	0.1091	0.1076
Research	0.0240	0.0237	0.0210



Comment: Ridge and Lasso coefficients are almost identical to OLS -- regularization barely changes anything, which tells us the OLS fit was not overfitting to begin with.

4. Testing the Assumptions of Linear Regression (50 points)

4.1 Multicollinearity check — VIF (drop iteratively until all VIF ≤ 5)

```
def compute_vif(X_df):
    if HAS_SM:
        Xc = sm.add_constant(X_df)
        vifs = pd.Series(
            [variance_inflation_factor(Xc.values, i) for i in range(1, Xc.shape[1])],
            index=X_df.columns
        )
    else:
        vifs = {}
        for col in X_df.columns:
            others = X_df.drop(columns=[col])
            lr = LinearRegression().fit(others, X_df[col])
            r2_i = lr.score(others, X_df[col])
            vifs[col] = 1 / (1 - r2_i) if r2_i < 1 else np.inf
        vifs = pd.Series(vifs)
    return vifs.sort_values(ascending=False)

X_vif = X_train.copy()
step = 1
while True:
    v = compute_vif(X_vif)
    print(f"--- VIF step {step} ---")
    print(v.round(3))
    if v.max() <= 5 or X_vif.shape[1] <= 1:
        break
    drop_col = v.idxmax()
    print(f"Dropping '{drop_col}' (VIF={v.max():.2f})\n")
    X_vif = X_vif.drop(columns=[drop_col])
    step += 1

final_features = list(X_vif.columns)
print("\nFinal feature set (all VIF <= 5):", final_features)
```

```
--- VIF step 1 ---
CGPA          4.655
GRE Score     4.490
TOEFL Score   3.664
SOP           2.786
University Rating
LOR           1.978
Research      1.518
```

```
dtype: float64
```

```
Final feature set (all VIF <= 5): ['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR', 'CGPA', 'Research']
```

Comment: All 7 predictors already have $VIF \leq 5$ on the first pass (highest is CGPA ≈ 4.66) -- no features need to be dropped. Correlation among GRE/TOEFL/CGPA (seen earlier) is not severe enough to destabilize the coefficients.

4.2 Final model on VIF-pruned features (here, same 7 features)

```
X_train_f, X_test_f = X_train[final_features], X_test[final_features]

if HAS_SM:
    X_train_f_sm = sm.add_constant(X_train_f)
    final_model = sm.OLS(y_train, X_train_f_sm).fit()
    print(final_model.summary())
    resid_final = final_model.resid.values
    yhat_final = final_model.fittedvalues.values
else:
    final_summary, r2_final, adjr2_final, f_final, fp_final, resid_final, yhat_final = manual_ols(X_train_f, y_train)
    print(final_summary.round(4))
```

OLS Regression Results

```
=====
Dep. Variable:      Chance of Admit      R-squared:                0.821
Model:              OLS                  Adj. R-squared:           0.818
Method:              Least Squares        F-statistic:             257.0
Date:                Wed, 12 Aug 2026      Prob (F-statistic):       3.41e-142
Time:                18:50:44              Log-Likelihood:          561.91
No. Observations:    400                  AIC:                    -1108.
Df Residuals:        392                  BIC:                    -1076.
Df Model:             7
Covariance Type:     nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	-1.4214	0.123	-11.549	0.000	-1.663	-1.179
GRE Score	0.0024	0.001	4.196	0.000	0.001	0.004
TOEFL Score	0.0030	0.001	3.174	0.002	0.001	0.005
University Rating	0.0026	0.004	0.611	0.541	-0.006	0.011
SOP	0.0018	0.005	0.357	0.721	-0.008	0.012
LOR	0.0172	0.005	3.761	0.000	0.008	0.026
CGPA	0.1125	0.011	10.444	0.000	0.091	0.134
Research	0.0240	0.007	3.231	0.001	0.009	0.039

```
=====
Omnibus:            86.232      Durbin-Watson:           2.050
Prob(Omnibus):       0.000      Jarque-Bera (JB):        190.099
Skew:                -1.107     Prob(JB):                 5.25e-42
Kurtosis:            5.551      Cond. No.                 1.37e+04
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.37e+04. This might indicate that there are strong multicollinearity or other numerical problems.

4.3 Mean of residuals ≈ 0

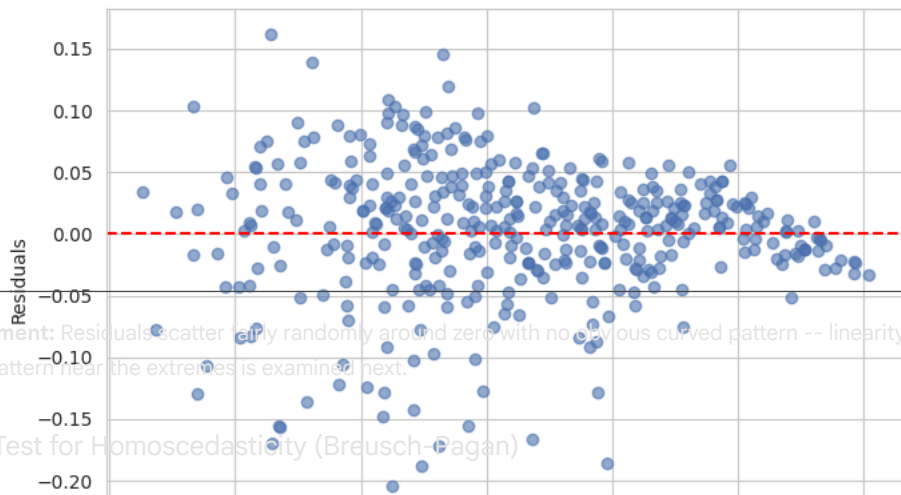
```
print("Mean of residuals:", resid_final.mean())
```

```
Mean of residuals: 3.9204750555707584e-16
```

4.4 Linearity of variables (no pattern in residual plot)

```
plt.figure(figsize=(7, 5))
plt.scatter(yhat_final, resid_final, alpha=0.6, color="#4C72B0")
plt.axhline(0, color="red", linestyle="--")
plt.xlabel("Fitted values")
plt.ylabel("Residuals")
plt.title("Residuals vs Fitted Values")
plt.tight_layout()
plt.show()
```

Residuals vs Fitted Values



Comment: Residuals scatter fairly randomly around zero with no obvious curved pattern -- linearity assumption reasonably holds. A mild fan pattern near the extremes is examined next.

4.5 Test for Homoscedasticity (Breusch-Pagan)

```
if HAS_SM:
    bp_stat, bp_p, _, _ = het_breuschpagan(resid_final, sm.add_constant(X_train_f))
else:
    def breusch_pagan(resid, X_df):
        n = len(resid)
        resid2 = resid ** 2
        Xd = np.column_stack([np.ones(n), X_df.values])
        beta_bp, _, _, _ = np.linalg.lstsq(Xd, resid2, rcond=None)
        fitted_bp = Xd @ beta_bp
        ss_res_bp = np.sum((resid2 - fitted_bp) ** 2)
        ss_tot_bp = np.sum((resid2 - resid2.mean()) ** 2)
        r2_bp = 1 - ss_res_bp / ss_tot_bp
        lm_stat = n * r2_bp
        p_val = 1 - stats.chi2.cdf(lm_stat, Xd.shape[1] - 1)
        return lm_stat, p_val
    bp_stat, bp_p = breusch_pagan(resid_final, X_train_f)

print(f"Breusch-Pagan LM stat = {bp_stat:.3f}, p-value = {bp_p:.4g}")

Breusch-Pagan LM stat = 25.156, p-value = 0.000712
```

Comment: $p < 0.05 \rightarrow$ we reject homoscedasticity; there is mild heteroscedasticity. This is common for a bounded 0-1 target (variance naturally shrinks near 0/1 and widens in the middle). It mainly affects the reliability of standard errors/p-values, not the coefficients themselves. A recommended fix for production: use robust (HC3) standard errors or a logit/beta-regression transform of the target.

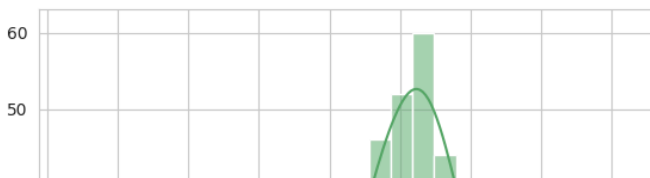
4.6 Normality of residuals

```
shapiro_stat, shapiro_p = stats.shapiro(resid_final)
print(f"Shapiro-Wilk stat = {shapiro_stat:.4f}, p-value = {shapiro_p:.4g}")

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sns.histplot(resid_final, kde=True, ax=axes[0], color="#55A868")
axes[0].set_title("Distribution of Residuals")
stats.probplot(resid_final, dist="norm", plot=axes[1])
axes[1].set_title("Q-Q Plot of Residuals")
plt.tight_layout()
plt.show()
```

Shapiro-Wilk stat = 0.9291, p-value = 7.735e-13

Distribution of Residuals



Q-Q Plot of Residuals

